

gharc: A stream-and-filter tool for the GitHub Archive on consumer hardware

Arav Panwar ¹

¹ Independent Researcher

DOI: [N/A](#)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: 01 January 1970

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

gharc is a command-line tool and Python library for extracting filtered subsets of the GitHub Archive (GHArchive) ([Grigorik, 2012](#)) on a personal computer. GHArchive records nearly every public GitHub event since 2011 and publishes it as hourly gzipped JSON-lines files; a single year is hundreds of gigabytes compressed and the full archive is several petabytes uncompressed. Most studies need only a slice of this, a few repositories or a few event types over a window of months, but the archive offers no server-side filter, so a researcher must obtain whole hours and discard almost all of them. gharc performs that selection as a stream: it downloads each hour to a temporary file, decompresses and filters it on the fly, writes only the matching events to Parquet or JSONL, and deletes the temporary file before moving on. Peak local storage is therefore set by the number of downloads in flight, one temporary file per worker, rather than by the length of the window processed. The intended users are software-engineering researchers, students, and small teams who need event-level GitHub data at multi-year scales but do not have a cloud-warehouse account or institutional storage.

Statement of need

The GitHub Archive is one of the most widely used data sources in mining software repositories (MSR) research. Its scale makes even modest studies awkward without institutional infrastructure, and the three common ways to work with it each impose a cost that excludes part of the research community. Bulk local downloads require enough disk to hold the raw hours before any filtering, which for a multi-month study exceeds a typical laptop. Cloud-warehouse mirrors on BigQuery and Snowflake ([GH Archive contributors, 2024](#)) are fast but require a billing account. Shared research infrastructures remove the storage problem but introduce an access dependency: an account, a remote job queue, or a maintained service that may lapse.

gharc targets the case these options leave open: a researcher on a personal machine, with no cloud account and bounded disk, who needs a specific slice of the archive over a long time range. By filtering each hour during the stream and keeping only the result, it makes the working storage independent of the window length, so the same laptop that can process one day can process a year. The motivating study was a six-month analysis of Apache Spark contributor activity ([Panwar, 2025](#)) whose original pipeline stored each month before filtering and peaked near 100 GB of intermediate disk; gharc reproduces that kind of analysis with bounded disk and a single command.

State of the field

GHArchive is the data source gharc consumes rather than a competitor. Among the tools that help researchers turn it, or related GitHub data, into analyzable subsets, the useful distinction

is the access model each assumes.

Cloud-warehouse mirrors of GHArchive on BigQuery and Snowflake ([GH Archive contributors, 2024](#)) offer SQL over the full dataset but require a cloud billing account. GHTorrent ([Gousios, 2013](#)) was for years the default GitHub-mining database, offering periodic dumps and a queryable mirror; its data collection has not been actively maintained since around 2019, with the most recent dumps from 2021, which illustrates how centrally hosted mirrors can lapse. Boa ([Dyer et al., 2013](#)) provides a domain-specific language for ultra-large-scale repository queries against curated datasets, accessed through a hosted service that requires account registration and approval. World of Code ([Ma et al., 2019](#)) gives researchers a curated cross-reference of millions of repositories through accounts and jobs submitted to dedicated servers. These shared infrastructures are powerful for cross-project queries at a scale a laptop cannot reach, but each adds an external dependency. PyDriller ([Spadini et al., 2018](#)) works at a different layer, mining commits and diffs from cloned git repositories rather than the GitHub event stream, and is complementary to gharc.

gharc occupies the remaining position: local-first, no account, no query service, and no schema-bound language, just the original GHArchive files streamed and filtered on demand. That position is a deliberate trade-off, described next.

Software design

gharc is built around one decision: decouple peak local storage from the time range by treating each hour as transient. The user supplies a date range and optional repository and event-type filters; gharc builds the GHArchive URL for each hour and dispatches the hours to a worker pool. Each worker downloads an hour, filters it line by line, and returns the matches; the main thread writes them out and the temporary file is removed ([Figure 1](#)).

The central trade-off follows from that decision. Streaming and filtering re-pays download bandwidth on every run, because nothing is cached locally for reuse, and it deliberately offers no cross-time query capability. In exchange, peak disk stays bounded and the tool needs no database, warehouse, or persistent local dataset. For the target setting, repeated reads of the same window are rare and disk is the binding constraint, so the design favors bounded storage over reuse, and leaves aggregation and joins to whatever the researcher already uses (pandas, Polars, DuckDB, Spark) over the small filtered output.

Three further design choices address correctness and robustness rather than storage.

Filtering before parsing. Within each hour a byte-level token check rejects lines that cannot match before any JSON parsing; only surviving lines are parsed and checked against the structured filter. The token check is a superset test, so it may admit a false positive that the structured filter then rejects, but it never discards a true match. On a selective filter this skips the large majority of lines for the cost of a substring scan; on a wide or empty filter it saves nothing and the run pays full parsing cost, which is the honest boundary of the optimization.

Stable columnar output across heterogeneous events. GitHub events share a top-level shape but differ in practice: the org field appears only for organization-owned repositories, and nested fields vary by event type. Writing Parquet incrementally over a stream turns this into a correctness issue, because a schema inferred from the first batch will reject or silently drop later batches whose columns differ. gharc pins an explicit top-level schema and JSON-stringifies nested fields, so every appended batch conforms regardless of which event types or ownership patterns occur in a given hour, and a run produces one file that reads back as a single table. This was a design lesson: an earlier first-batch-inference approach corrupted output on mixed-ownership runs.

Failure semantics for unreliable connections. Because the target setting is residential internet, gharc separates a download that failed from an hour that genuinely had no matches. A persistent download failure is reported with a non-zero exit rather than recorded as an empty

hour, so a flaky link cannot silently yield an incomplete dataset, while a genuinely absent archive hour is treated as zero events. Completed hours are recorded in a sidecar checkpoint fingerprinted by the run parameters, so an interrupted run resumes the remaining hours instead of restarting. Resume is supported for JSONL output, since a closed Parquet file cannot be appended to; the tool makes that asymmetry explicit and recommends JSONL with a later conversion step for long runs.

Two scope boundaries shape how the tool should be used. It is bound by HTTPS download throughput rather than local CPU, so adding workers beyond a few gives diminishing returns once a residential connection saturates, and matching events for an hour are buffered before writing, so working memory stays near 100 MB for selective filters but grows for very wide ones. GHArchive also uses the GitHub Events API schema from January 2015 onward and an older Timeline API schema before that; gharc does not normalize across that boundary, so studies spanning 2011 to 2014 should expect some fields to be missing or shaped differently.

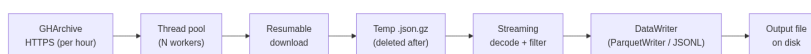


Figure 1: Stream-and-filter architecture.

Research impact statement

gharc was motivated by, and reproduces, a six-month study of Apache Spark contributor activity (Panwar, 2025) originally run as an undergraduate project. The earlier pipeline downloaded GHArchive month by month, filtered each month with a separate script, peaked near 100 GB of intermediate disk, and contained a date off-by-one error in which an inclusive end bound pulled in an extra month. gharc removes both problems: storage stays bounded and the date range is end-exclusive by construction.

More broadly, the tool lowers the entry cost for event-level GitHub research to a laptop and an internet connection, the configuration available to students and independent researchers without cloud quotas. On a Windows laptop over a residential connection, a six-hour window filtered to apache/spark completed in under 80 seconds and recovered the same 14 events at both one and four workers, with peak local disk held to roughly 85 MB and 250 MB respectively; the reproducible benchmark scripts are included in the repository. The package is distributed on PyPI with an automated test suite, and tagged releases are archived on Zenodo, so analyses that depend on it can pin and cite a fixed version.

AI usage disclosure

Author to complete before submission. JOSS requires a truthful statement here disclosing any use of generative AI in the software, its documentation, or this paper. If no such tools were used, that must be stated explicitly.

Software availability

The source code is hosted at github.com/aravpanwar/gharc under the MIT license. Tagged releases are archived on Zenodo; the concept DOI [10.5281/zenodo.19814232](https://doi.org/10.5281/zenodo.19814232) always resolves to the latest archived version.

Acknowledgements

The author thanks Ilya Grigorik and the GHArchive maintainers for the public dataset on which this tool depends, and the maintainers of the requests (Reitz, 2011), pandas (McKinney,

2010), pyarrow (Apache Arrow contributors, 2024), tqdm, and orjson libraries that gharc builds on.

References

- Apache Arrow contributors. (2024). *Apache arrow*. <https://arrow.apache.org/>
- Dyer, R., Nguyen, H. A., Rajan, H., & Nguyen, T. N. (2013). Boa: A language and infrastructure for analyzing ultra-large-scale software repositories. *Proceedings of the 2013 International Conference on Software Engineering*, 422–431.
- GH Archive contributors. (2024). *GH Archive on Google BigQuery*. <https://www.gharchive.org/#bigquery>.
- Gousios, G. (2013). The GHTorrent dataset and tool suite. *Proceedings of the 10th Working Conference on Mining Software Repositories*, 233–236.
- Grigorik, I. (2012). *GH Archive: A GitHub timeline archive*. <https://www.gharchive.org/>.
- Ma, Y., Bogart, C., Amreen, S., Zaretski, R., & Mockus, A. (2019). World of code: An infrastructure for mining the universe of open source VCS data. *2019 IEEE/ACM 16th International Conference on Mining Software Repositories*, 143–154.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51–56.
- Panwar, A. (2025). *Apache Spark codebase evolution: A six-month GHArchive analysis*. https://github.com/aravpanwar/Spark_Codebase_Evolution.
- Reitz, K. (2011). *Requests: HTTP for humans*. <https://requests.readthedocs.io>
- Spadini, D., Aniche, M., & Bacchelli, A. (2018). PyDriller: Python framework for mining software repositories. *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 908–911.